Relatório de Arquitetura: Sistemas, Histórias de Usuário e Padrões (ADRs)

Sistema: SYS01

Propósito: Suporte multiárea de acordo com normativos e procedimentos da empresa

História de Usuário: SYS01-US01

"Eu como atendente de loja quero tirar dúvidas sobre o procedimento adequado para abordar a solicitação do cliente;"

Critérios de Aceite:
[SYS01-US01-AC01] O sistema deverá responder ao atendente de acordo com os documentos oficiais da empresa;
[SYS01-US01-AC02] O sistema deverá informar o atendente que não pode ajudar, caso nenhum documento contenha a orientação para o procedimento.

Padrões e ADRs Aplicáveis:

Padrão [1]: Encapsulando modelos de ML em salvaguardas baseadas em regras

Motivação: É impossível garantir a correção das previsões de modelos de ML, portanto elas não devem ser usadas diretamente em funções relacionadas à segurança ou à proteção. Além disso, os modelos de ML podem ser instáveis e vulneráveis a ataques adversariais, ruído nos dados e deriva.

Solução: Introduzir um mecanismo determinístico, baseado em regras, que decide o que fazer com os resultados das previsões, por exemplo, com base em verificações adicionais de qualidade.

Consequências: Risco reduzido de impactos negativos de previsões incorretas, mas uma arquitetura mais complexa.

ADR: Salvaguardas para respostas fundamentadas em documentos Proposto

Contexto: Os atendentes de loja precisam de orientações sobre procedimentos para lidar com solicitações de clientes. O sistema deve

responder de acordo com os documentos oficiais da empresa e declarar que não pode ajudar quando nenhum documento contiver orientações para o procedimento. As respostas produzidas por IA nem sempre satisfazem essas condições, portanto a arquitetura deve impedir que orientações sem respaldo cheguem aos atendentes.

Decisão: Colocaremos uma salvaguarda determinística, baseada em regras, entre as orientações produzidas por IA e sua entrega aos atendentes de loja. A salvaguarda permitirá uma resposta apenas quando ela for consistente com as orientações contidas nos documentos oficiais da empresa. Se nenhum documento oficial contiver orientações para o procedimento solicitado, o sistema dirá ao atendente que não pode ajudar.

Consequências da Decisão: Isso reduzirá o risco de os atendentes receberem orientações sobre procedimentos sem respaldo, embora não possa garantir que toda resposta permitida esteja correta. A salvaguarda acrescenta complexidade arquitetural e pode impedir a entrega de uma resposta quando as orientações produzidas por IA não atenderem às suas regras.

Padrão [16]: Rejeição Parcial da Responsabilidade pela Segurança

Motivação: Desenvolvimentos recentes em ML e DL permitiram que sistemas probabilísticos com grandes espaços de entrada e saída fossem explorados em sistemas críticos para a segurança. Isso introduz desafios relacionados à segurança, como: (1) Complexidade e opacidade do modelo, (2) Lidar com saídas probabilísticas, (3) Sensibilidade a mudanças de distribuição (4) A verificação formal é impossível ou não é escalável e outros. Essa responsabilidade pela segurança tem de ser tratada.

Solução: Um sistema decide rejeitar as previsões de um algoritmo de DL ou adiar quaisquer decisões até estar confiante o suficiente

Consequências: Pode-se permitir alguma incerteza em um sistema, ao mesmo tempo em que se impõem restrições relativamente baixas aos algoritmos de DL usados.

ADR: Rejeição de Respostas Procedimentais Sem Respaldo

Proposto

Contexto: Os atendentes de loja precisam de orientação sobre como lidar com as solicitações dos clientes. O sistema deve responder de acordo com os documentos oficiais da empresa e informar o atendente quando

nenhum documento contiver orientação para o procedimento solicitado. A arquitetura deve, portanto, levar em conta solicitações que não têm uma resposta documentada.

Decisão: Rejeitaremos uma resposta procedimental proposta quando os documentos oficiais da empresa não a respaldarem. Se nenhum documento contiver orientação para o procedimento solicitado, diremos ao atendente que o sistema não pode ajudar, em vez de fornecer uma resposta sem respaldo.

Consequências da Decisão: Os atendentes não receberão orientações procedimentais que não tenham respaldo nos documentos oficiais. As solicitações sem orientação documentada permanecerão sem resposta, de modo que a capacidade do sistema de ajudar depende da orientação contida nesses documentos.

Padrão [60]: Padrão de disponibilização baseado em parâmetros

Motivação: Para controlar a predição com parâmetro. Quando é possível controlar com base em regras.

Solução: Adicione um mecanismo em seu código para alterar o comportamento de todas ou de uma parte das predições, incluindo parada, novas tentativas ou tempo limite. Como é difícil manter alta acurácia sem nenhum erro em um modelo para todos os casos ou todos os dados de entrada, é melhor estimar possíveis anomalias e controlar com um mecanismo baseado em regras.

Consequências: Prós: evitar predições anormais em casos extremos. Contras: Impossível cobrir todos os casos. O aumento das regras gera complexidade na operação.

ADR: Controle de Respostas Fundamentadas em Documentos

Proposto

Contexto: Os atendentes de loja precisam de respostas sobre como lidar com solicitações de clientes. O sistema deve responder de acordo com documentos oficiais da empresa e deve dizer que não pode ajudar quando nenhum documento contém orientações para o procedimento. A arquitetura, portanto, precisa controlar quando fornece orientações procedimentais.

Decisão: Aplicaremos um controle baseado em regras às respostas procedimentais. Forneceremos orientações apenas quando um

documento oficial da empresa contiver orientações para o procedimento e manteremos a resposta consistente com esse documento. Quando nenhum documento contiver tais orientações, não forneceremos orientações procedimentais e diremos ao atendente que o sistema não pode ajudar.

Consequências da Decisão: Esse controle reduz o risco de dar aos atendentes orientações procedimentais não respaldadas por documentos oficiais. Sua eficácia depende de determinar corretamente se os documentos contêm orientações aplicáveis. O controle não pode contemplar todos os casos, e regras adicionais podem aumentar a complexidade operacional.

História de Usuário: SYS01-US02

"Eu como atendente de TI quero prestar o suporte de primeiro nível para os usuários da empresa, tais como bloqueios de sites ou instalação de softwares"

Critérios de Aceite:
[SYS01-US02-AC01] O sistema deverá registrar um chamado para documentação da solicitação por meio de integração com o sistema de chamados;
[SYS01-US02-AC02] O sistema deverá responder os procedimentos e dúvidas informados pelo usuário no âmbito do portfólio dos serviços de TI;

Padrões e ADRs Aplicáveis:

Padrão [3]: Cliente-Servidor

Motivação: Um número de usuários/aplicações requer dados de uma aplicação.

Solução: Um componente tem o papel de servidor e pelo menos um componente tem o papel de cliente, iniciando conexões a fim de obter algum serviço.

Consequências: Os dados, assim como os periféricos de rede, são controlados centralmente. Uma desvantagem, porém, é que o servidor é caro para adquirir e gerenciar.

ADR: Integração cliente-servidor para registro de chamados

Proposto

Contexto: O atendimento de primeiro nível de TI abrange solicitações como bloqueio de sites e instalação de softwares. O sistema deve responder a procedimentos e dúvidas sobre o portfólio de serviços de TI e documentar as solicitações por meio de integração com o sistema de chamados. Essa integração exige definir qual sistema solicita o registro e qual presta esse serviço.

Decisão: Vamos estabelecer uma relação cliente-servidor para o registro de chamados. O sistema de suporte atuará como cliente e iniciará solicitações ao sistema de chamados, que atuará como servidor e prestará o serviço de registro para documentar as solicitações dos usuários.

Consequências da Decisão: O registro dos chamados ficará centralizado no sistema de chamados. Em contrapartida, a documentação das solicitações dependerá da disponibilidade desse sistema, e a operação do serviço de registro exigirá gestão. Essa integração não define como o sistema responderá às dúvidas e aos procedimentos do portfólio de TI.

História de Usuário: SYS01-US03

"Eu como usuário desejo que o sistema mostre todas as fontes usadas nas respostas solicitadas"

Critérios de Aceite:
[SYS01-US03-AC01] O sistema deverá informar o documento, versão e seção do trecho recuperado
[SYS01-US03-AC02] O sistema deverá informar apenas o que está no catálogo da empresa

Padrões e ADRs Aplicáveis:

Padrão [1]: Encapsulando modelos de ML em salvaguardas baseadas em regras

Motivação: É impossível garantir a correção das previsões dos modelos de ML, portanto elas não devem ser usadas diretamente em funções relacionadas à segurança operacional ou à segurança da informação. Além disso, os modelos de ML podem ser instáveis e vulneráveis a ataques adversariais, ruído nos dados e deriva.

Solução: Introduzir um mecanismo determinístico, baseado em regras, que decida o que fazer com os resultados das previsões, por exemplo, com base em verificações adicionais de qualidade.

Consequências: Risco reduzido de impactos negativos de previsões incorretas, mas uma arquitetura mais complexa.

> **ADR: Salvaguardas baseadas em regras para respostas fundamentadas no catálogo** Proposto
>
> **Contexto:** O sistema de suporte de múltiplas áreas deve mostrar todas as fontes usadas em uma resposta, identificando o documento, a versão e a seção de cada trecho recuperado. As respostas devem conter apenas informações do catálogo da empresa. Como o conteúdo gerado por IA pode não atender de forma confiável a essas condições, a arquitetura precisa de uma maneira de verificar as respostas antes de apresentá-las.
>
> **Decisão:** Colocaremos verificações determinísticas, baseadas em regras, entre a geração e a apresentação das respostas. Essas verificações permitirão apenas conteúdo sustentado por trechos recuperados do catálogo da empresa e exigirão que a resposta mostre todas as fontes usadas, com o documento, a versão e a seção de cada uma. O sistema não apresentará conteúdo que não atender a essas verificações.
>
> **Consequências da Decisão:** As respostas apresentadas serão mais rastreáveis ao catálogo da empresa, reduzindo o risco de informações sem respaldo. As verificações acrescentarão complexidade arquitetural e poderão impedir que algum conteúdo seja mostrado quando não for possível estabelecer o respaldo no catálogo ou os detalhes exigidos das fontes.

Padrão [13]: Arquitetura Fluida

Motivação: Padrões clássicos de arquitetura e UX, embora amplamente bem-sucedidos, sofrem das seguintes limitações em contextos de AI: (1) Interfaces estáticas, (2) Falta de bidirecionalidade, (3) Atualizações lentas, (4) personalização subótima, (5) interpretabilidade e explicabilidade

Solução: Além de qualquer AI que execute no backend, há uma AI no frontend para evoluir a interface e adaptá-la a mudanças. Além disso, há um orquestrador de AI. Seus propósitos são monitorar o desempenho tanto da AI do frontend quanto da AI do backend para garantir que seus respectivos desempenhos sejam satisfatórios e testemunhar negociações de contratos para ver como as interações entre o frontend e o backend evoluem ao longo do tempo.

Consequências: As limitações mencionadas anteriormente são atenuadas.

ADR: Apresentação Adaptativa das Fontes das Respostas Proposto

Contexto: Os usuários precisam ver todas as fontes usadas em uma resposta solicitada, incluindo o documento, a versão e a seção de cada trecho recuperado. O sistema deve relatar apenas informações contidas no catálogo da empresa. A apresentação das respostas, portanto, precisa permanecer alinhada às informações de suas fontes.

Decisão: Estabeleceremos uma AI de frontend que adapta a apresentação das respostas para mostrar todas as fontes usadas e o documento, a versão e a seção de cada uma. Uma AI de backend fornecerá respostas limitadas ao catálogo da empresa e às informações das fontes associadas. Um orquestrador de AI monitorará ambas as capacidades de AI e supervisionará como a interação entre elas evolui, para que a apresentação permaneça alinhada às respostas e às suas fontes.

Consequências da Decisão: Os usuários poderão ver as fontes do catálogo por trás das respostas, tornando suas origens mais fáceis de entender. O frontend e o backend precisarão manter o conteúdo das respostas e os detalhes das fontes alinhados. A AI adicional de frontend e o orquestrador também introduzirão responsabilidades de monitoramento e coordenação.

Padrão [60]: Padrão de disponibilização baseado em parâmetros

Motivação: Para controlar a predição com parâmetro. Quando é possível controlar com base em regras.

Solução: Adicione um mecanismo no seu código para alterar o comportamento de todas as predições ou de parte delas, incluindo parada, nova tentativa ou tempo limite. Como é difícil manter alta acurácia sem nenhum erro para um modelo em todos os casos ou para todos os dados de entrada, é melhor estimar possíveis anomalias e controlar com um mecanismo baseado em regras.

Consequências: Prós: evitar predições anormais para casos de borda. Contras: impossível cobrir todos os casos. O aumento no número de regras gera complexidade na operação.

ADR: Controles Baseados em Regras para Respostas do Catálogo

Proposto

Contexto: Os usuários precisam ver todas as fontes usadas em uma resposta, incluindo o documento, a versão e a seção de cada trecho recuperado. As respostas devem relatar apenas informações do catálogo da empresa. Portanto, a arquitetura precisa controlar quais informações aparecem nas respostas, preservando os detalhes exigidos das fontes.

Decisão: Aplicaremos controles baseados em regras às respostas para que relatem apenas conteúdo do catálogo da empresa. Para cada trecho recuperado usado em uma resposta, mostraremos seu documento, versão e seção; conteúdo que não atender a essas condições não será relatado.

Consequências da Decisão: As respostas ficarão limitadas ao conteúdo do catálogo, e os usuários poderão identificar as fontes usadas. Conteúdo sem os detalhes exigidos das fontes talvez tenha de ser excluído. Os controles não podem cobrir todas as possíveis respostas errôneas, e a manutenção de mais regras pode aumentar a complexidade operacional.

Sistema: SYS02

Propósito: Assistente para geração de documentos licitatórios

História de Usuário: SYS02-US01

"Eu como usuário desejo que o sistema gere um documento inicial de acordo com o registro cadastrado no sistema de licitações"

Critérios de Aceite:
[SYS02-US01-AC01] O sistema deverá seguir o layout previamente cadastrado pelo administrador do sistema
[SYS02-US01-AC02] O sistema deverá substituir as variáveis com o conteúdo gerado pelo modelo de IA
[SYS02-US01-AC03] O sistema deverá utilizar o layout apropriado para o tipo de documento
[SYS02-US01-AC04] O sistema deverá utilizar o padrão de linguagem na norma técnico-jurídica
[SYS02-US01-AC05] O sistema deverá concatenar o texto do layout com os textos das variáveis mantendo um padrão legível e harmônico

Padrões e ADRs Aplicáveis:

Padrão [2]: Tubos e Filtros

Motivação: Um sistema precisa executar uma variedade de tarefas de diferentes níveis de complexidade sobre os dados que processa.

Solução: Os dados são processados por meio de uma série de procedimentos, que têm diferentes entradas em várias etapas e produzem resultados incrementais à medida que ocorre a comparação entre as imagens de consulta e a imagem de entrada.

Consequências: O sistema se torna mais flexível; remover, modificar e adicionar filtros é mais fácil do que em um sistema monolítico

ADR: Geração em Etapas de Documentos de Contratação

Proposto

Contexto: O sistema deve gerar um documento inicial com base em um registro no sistema de contratações. Deve usar o layout cadastrado pelo administrador adequado ao tipo de documento, substituir as variáveis do layout por conteúdo gerado por AI e produzir um texto legível e harmonioso em linguagem técnico-jurídica. Esses requisitos envolvem transformações distintas que devem funcionar em conjunto no documento final.

Decisão: Geraremos o documento por meio de uma sequência de etapas de processamento. As etapas selecionarão o layout cadastrado adequado, substituirão suas variáveis por conteúdo gerado por AI e combinarão o layout e o texto das variáveis em um documento legível e harmonioso em linguagem técnico-jurídica. Cada etapa passará seu resultado para a próxima.

Consequências da Decisão: Separar as transformações tornará mais fácil ajustar uma etapa sem alterar todo o fluxo de geração. As etapas também precisarão permanecer coordenadas: uma alteração em um resultado intermediário pode afetar a legibilidade, a formatação ou a linguagem do documento final.

Padrão [4]: Modelo-Visão-Controlador (MVC)

Motivação: O sistema deve facilitar múltiplas visões do mesmo objeto de dados e uma única visão de múltiplos objetos de dados.

Solução: Inclui três partes: Modelo (dados internos), Visão (representação dos dados) e Controlador (Controlador de I/O). Apenas a camada do modelo deve ser alterada.

Consequências: Esse padrão aumenta muito a reutilização do código e a expansibilidade do sistema.

> **ADR: Separar o conteúdo do documento do layout e do controle da geração** Proposto
>
> **Contexto:** O sistema deve gerar um documento inicial de aquisição a partir de um registro cadastrado. Ele deve usar o layout cadastrado pelo administrador apropriado ao tipo de documento, substituir as variáveis do layout por conteúdo gerado por IA e produzir um texto legível e coerente em linguagem técnico-jurídica. Esses requisitos distinguem o conteúdo do documento de sua representação e do tratamento da solicitação de geração.
>
> **Decisão:** Separaremos a geração do documento em um modelo que contém o registro cadastrado e o conteúdo gerado para as variáveis, uma visão que representa o documento usando o layout cadastrado apropriado e um controlador que coordena a solicitação de geração e o documento resultante. A visão combinará o texto do layout com o conteúdo das variáveis substituídas para produzir um documento legível e coerente no estilo técnico-jurídico exigido.
>
> **Consequências da Decisão:** Separar o conteúdo de sua representação facilitará a reutilização do comportamento de geração entre tipos de documento e seus layouts. Isso também exigirá que o modelo, a visão e o controlador permaneçam coordenados à medida que os layouts e os tipos de documento mudem. O texto concluído ainda deve ser verificado quanto à legibilidade, à coerência e à linguagem técnico-jurídica; a separação, por si só, não garante essas qualidades.

Padrão [14]: Distinguir a lógica de negócios do modelo de ML

Motivação: Os sistemas de aprendizado de máquina (ML) são complexos porque seus componentes de ML devem ser (re)treinados regularmente e têm um comportamento não determinístico intrínseco. Assim como em outros sistemas, os requisitos de negócios para esses sistemas, bem como os algoritmos de ML, mudam ao longo do tempo.

Solução: Defina APIs claras entre componentes tradicionais e de ML. Coloque componentes de negócios e de ML com diferentes responsabilidades em três camadas. Divida os fluxos de dados em três.

Consequências: Desacoplar os componentes de negócios “tradicionais” dos componentes de ML permite que os componentes de ML sejam monitorados

e ajustados para atender aos requisitos dos usuários e às entradas em mudança.

ADR: Separação entre composição documental e geração por IA

Proposto

Contexto: O documento inicial deve corresponder ao registro cadastrado no sistema de licitações, usar o layout apropriado ao tipo de documento e seguir o layout cadastrado pelo administrador. O conteúdo das variáveis será gerado pelo modelo de IA em linguagem técnico-jurídica, enquanto a substituição das variáveis e a composição do texto devem preservar a legibilidade e a harmonia do documento. Essas responsabilidades precisam permanecer distinguíveis.

Decisão: Separaremos a lógica de negócios que seleciona o layout registrado, substitui variáveis e compõe o documento do modelo de AI que gera o conteúdo para essas variáveis. Definiremos uma interface clara por meio da qual o conteúdo gerado será fornecido para inserção no layout apropriado, preservando a linguagem técnico-jurídica exigida e um resultado legível e harmonioso.

Consequências da Decisão: As regras de layout e composição podem ser mantidas separadamente dos ajustes no conteúdo gerado por AI. Essa separação torna mais claro qual responsabilidade abordar quando o documento não atende aos seus requisitos. Ela também exige a manutenção da interface entre as duas responsabilidades; separá-las não garante, por si só, que o texto gerado siga o padrão de linguagem exigido ou se encaixe harmoniosamente no layout.

Padrão [45]: Padrão Builder

Motivação: nan

Solução: Separar a construção de um objeto complexo de sua representação

Consequências: nan

ADR: Separar a construção do documento de sua representação de layout Proposto

Contexto: O sistema deve gerar um documento inicial de licitação com base em um registro cadastrado. Deve usar o layout cadastrado pelo administrador apropriado ao tipo de documento, substituir as variáveis do

layout por conteúdo gerado por AI e produzir um texto legível e harmonioso em linguagem técnico-jurídica. Isso exige que o conteúdo e o layout do documento sejam reunidos sem tratá-los como a mesma questão.

Decisão: Separaremos a construção do documento inicial de sua representação de layout. O layout cadastrado selecionado para o tipo de documento definirá a representação, enquanto a construção do documento substituirá suas variáveis por conteúdo gerado por AI e combinará esse conteúdo com o texto do layout em um documento legível e harmonioso em linguagem técnico-jurídica.

Consequências da Decisão: Os layouts podem variar conforme o tipo de documento sem serem confundidos com o conteúdo gerado por AI usado para preenchê-los. A construção do documento ainda deve considerar as variáveis de cada layout selecionado e garantir que o texto combinado permaneça coerente e siga o padrão de linguagem exigido; separar a construção da representação não garante essas qualidades por si só.

Padrão [50]: Padrão síncrono

Motivação: Quando, na sua lógica de negócios, a inferência do modelo é um bloqueio para prosseguir para a próxima etapa

Solução: bloqueia o fluxo de trabalho do sistema até que a predição termine

Consequências: Fácil de gerenciar com sua simplicidade. Todos os aspectos operacionais, como rastreamento de transações, monitoramento etc., também se tornam fáceis. O fluxo de trabalho do serviço se torna simples, pois o processo não prosseguirá até que a predição seja concluída. Mas: (1) A latência da predição pode se tornar um gargalo de desempenho. (2) Você talvez tenha que considerar uma solução alternativa para não degradar a experiência do usuário por causa da latência da predição. (3) Se o cliente do seu serviço for outro serviço, então esse padrão leva a threads bloqueadas no lado do cliente.

ADR: Montagem síncrona de documentos iniciais Proposto

Contexto: O sistema deve gerar um documento inicial de aquisição usando o layout registrado pelo administrador adequado ao seu tipo de documento. Ele deve substituir as variáveis do layout por conteúdo gerado por IA e combinar o texto de forma legível e harmoniosa usando

linguagem técnico-jurídica. A substituição de variáveis e a montagem do documento dependem de o modelo concluir sua geração.

Decisão: Faremos com que o fluxo de trabalho de geração de documentos aguarde o modelo de IA terminar de gerar o conteúdo das variáveis antes de substituir as variáveis e combinar esse conteúdo com o layout registrado apropriado. O fluxo de trabalho não montará o documento inicial até que o conteúdo gerado esteja disponível.

Consequências da Decisão: Isso mantém a sequência de geração e montagem simples e torna o fluxo de trabalho mais fácil de rastrear e monitorar. A latência do modelo atrasará a conclusão do documento e poderá afetar a experiência do usuário.

História de Usuário: SYS02-US02

"Eu como usuário desejo que o sistema melhore os textos informados, corrigindo eventuais erros e ajustando-o à norma técnico-jurídica"

Critérios de Aceite:
[SYS02-US02-AC01] O sistema deverá alterar os textos sem alterar o contexto ou o propósito do artefato produzido
[SYS02-US02-AC02] O sistema deverá aceitar diversos tipos de documentos previamente padronizados
[SYS02-US02-AC03] O sistema deverá permitir que o usuário reformule, resuma ou expanda o conteúdo com contexto;

Padrões e ADRs Aplicáveis:

Padrão [1]: Encapsulando modelos de ML em salvaguardas baseadas em regras

Motivação: É impossível garantir a correção das previsões de modelos de ML, portanto elas não devem ser usadas diretamente para funções relacionadas à segurança ou à proteção. Além disso, os modelos de ML podem ser instáveis e vulneráveis a ataques adversariais, ruído nos dados e deriva.

Solução: Introduzir um mecanismo determinístico, baseado em regras, que decida o que fazer com os resultados das previsões, por exemplo, com base em verificações adicionais de qualidade.

Consequências: Risco reduzido de impactos negativos de previsões incorretas, mas uma arquitetura mais complexa.

ADR: Salvaguardas baseadas em regras para revisões de documentos Proposto

Contexto: O assistente deve aprimorar o texto fornecido pelo usuário, incluindo correções e ajustes às convenções técnico-jurídicas. Também deve oferecer suporte a tipos de documentos previamente padronizados e permitir que os usuários reformulem, resumam ou expandam o conteúdo. Essas alterações devem preservar o contexto e a finalidade do artefato resultante, ao passo que revisões automatizadas podem produzir resultados incorretos.

Decisão: Colocaremos salvaguardas determinísticas, baseadas em regras, entre a revisão automatizada do texto e a aceitação do conteúdo revisado. As salvaguardas verificarão as correções, os ajustes, as reformulações, os resumos e as expansões propostos segundo condições para preservar o contexto e a finalidade do artefato, e impedirão que revisões que não passem nessas verificações sejam aplicadas.

Consequências da Decisão: As salvaguardas reduzirão o risco de aceitar revisões que alterem o contexto ou a finalidade de um artefato, mas não podem garantir que todas as revisões aceitas estejam corretas. Elas também tornarão a arquitetura mais complexa e poderão impedir algumas revisões aceitáveis quando essas revisões não passarem nas verificações.

Padrão [2]: Tubos e Filtros

Motivação: Um sistema precisa realizar uma variedade de tarefas de complexidade variável nos dados que processa.

Solução: Os dados são processados por meio de uma série de procedimentos, que têm diferentes entradas em várias etapas e produzem resultados incrementais à medida que ocorre a comparação entre as imagens de consulta e a imagem de entrada.

Consequências: O sistema se torna mais flexível; remover, modificar e adicionar filtros é mais fácil do que em um sistema monolítico

ADR: Pipeline de processamento de texto de documentos com preservação do contexto Proposto

Contexto: Os usuários precisam melhorar o texto em documentos padronizados de contratação pública, corrigindo erros e alinhando a redação às normas técnico-jurídicas. Eles também precisam reformular, resumir ou expandir o conteúdo sem alterar o contexto ou a finalidade do artefato resultante. Essas variadas tarefas de processamento de texto precisam funcionar em conjunto em diferentes tipos de documentos padronizados.

Decisão: Processaremos o texto dos documentos por meio de uma sequência de etapas de transformação separáveis. As etapas realizarão a correção, o ajuste técnico-jurídico e a reformulação, o resumo ou a expansão solicitados pelo usuário, conforme apropriado. Cada etapa passará seu resultado para a próxima, preservando o contexto do texto e a finalidade do artefato.

Consequências da Decisão: Separar as transformações tornará mais fácil modificar ou adicionar etapas de processamento de texto do que em um processo único e combinado. No entanto, as etapas precisarão funcionar em conjunto para que as mudanças sucessivas não alterem o contexto ou a finalidade do documento.

Padrão [60]: Padrão de disponibilização baseado em parâmetros

Motivação: Para controlar a predição com parâmetro. Quando é possível controlar com base em regras.

Solução: Adicione um mecanismo ao seu código para alterar o comportamento de todas as predições ou de parte delas, incluindo interrupção, nova tentativa ou tempo limite. Como é difícil manter alta precisão sem nenhum erro para um modelo em todos os casos ou para todos os dados de entrada, é melhor estimar uma possível anomalia e controlar com um mecanismo baseado em regras.

Consequências: Prós: evitar predição anormal para casos extremos. Contras: Impossível cobrir todos os casos. O aumento das regras gera complexidade na operação.

ADR: Transformação de documentos controlada por parâmetros

Proposto

Contexto: Os usuários precisam corrigir e adaptar textos às normas técnico-jurídicas em diferentes tipos de documentos previamente padronizados. Eles devem poder reformular, resumir ou expandir o conteúdo, preservando o contexto e o propósito do artefato resultante.

Essas escolhas de transformação exigem controle sobre o texto gerado, e uma revisão que, de outro modo, seria útil pode ser inaceitável se alterar o significado do artefato.

Decisão: Vamos parametrizar a transformação de texto de acordo com a ação selecionada pelo usuário — reformular, resumir ou expandir — e com o contexto e o propósito do documento. Vamos aplicar controles baseados em regras ao texto gerado e interromper ou tentar novamente uma transformação quando esses controles identificarem um desvio desse contexto ou propósito.

Consequências da Decisão: Os usuários podem solicitar diferentes transformações enquanto o sistema limita revisões que suas regras identificam como inconsistentes com o contexto ou o propósito do artefato. Os controles não conseguem detectar toda mudança problemática, portanto não garantem a preservação em todos os casos. Adicionar e manter regras também aumentará a complexidade operacional.

Padrão [64]: Arquitetura Daisy

Motivação: Exemplo: organizações adquirem a capacidade de escalar seus processos de produção de conteúdo premium por meio do uso de aprendizado de máquina, análise de texto e técnicas de anotação, e então estendem a cobertura desse ferramental ao máximo possível de sua geografia de conteúdo restante

Solução: Mudar de pipelines de produção de conteúdo que são a.) baseados em push; b.) manuais e c.) lineares; para ciclos de enriquecimento de conteúdo que são a.) baseados em pull; b.) automatizados; e c.) sob demanda e iterativos

Consequências: Resultado de três habilitadores técnicos: (1) Influência do Kanban no design de software (2) Ampliação em escala da análise automatizada de conteúdo e da criação de metadados ricos (3) Adoção de arquiteturas de microsserviços

ADR: Enriquecimento Iterativo de Texto sob Demanda Proposto

Contexto: Os usuários precisam melhorar o texto em tipos de documentos previamente padronizados, corrigindo erros e alinhando-o a normas técnico-jurídicas. Eles também devem poder solicitar reformulações, resumos e expansões sensíveis ao contexto sem alterar o

contexto ou a finalidade do artefato resultante. Um processo de edição fixo e unidirecional não acomodaria essas solicitações sucessivas.

Decisão: Estruturaremos a melhoria de texto como um ciclo de enriquecimento automatizado, iniciado pelo usuário e iterativo. Para cada solicitação, o sistema transformará o texto fornecido de acordo com a melhoria solicitada e permitirá novas reformulações, sumarizações ou expansões, preservando o contexto do texto e a finalidade do artefato nos tipos de documentos suportados.

Consequências da Decisão: Os usuários podem solicitar melhorias conforme necessário e refinar o resultado por meio de transformações sucessivas, em vez de depender de uma única etapa de edição. Transformações repetidas podem dificultar a preservação do contexto e da finalidade originais, particularmente entre diferentes tipos de documentos; cada iteração deve permanecer alinhada a esses requisitos.

Padrão [70]: Padrão de disponibilização baseada em condições

Motivação: Quando o alvo da predição varia muito conforme a condição. Quando é possível selecionar condicionalmente um modelo de maneira baseada em regras.

Solução: Implantar vários modelos e definir regras de seleção para eles. Dependendo de certos atributos dos dados de entrada, o modelo mais adequado é usado.

Consequências: Positivo: Selecionar um modelo adequado dependendo da condição. Negativo: O custo operacional aumenta com o número de modelos.

ADR: Seleção condicional de modelos para melhoria de texto em documentos Proposto

Contexto: Os usuários precisam melhorar o texto de documentos padronizados de contratação, corrigindo erros e alinhando-o às normas técnico-jurídicas. O sistema deve preservar o contexto e a finalidade do artefato, ao mesmo tempo que oferece suporte a três resultados solicitados diferentes: reformulação, resumo e expansão do conteúdo. O resultado esperado, portanto, varia conforme a operação solicitada pelo usuário.

Decisão: Disponibilizaremos vários modelos de melhoria de texto e estabeleceremos regras que selecionem um modelo conforme o usuário solicite reformulação, resumo ou expansão. O modelo selecionado

processará o texto no contexto de seu documento, com o objetivo de corrigir erros e atender à norma técnico-jurídica aplicável sem alterar o contexto ou a finalidade do artefato.

Consequências da Decisão: A seleção pela operação solicitada permite que o sistema direcione cada tarefa de melhoria de texto a um modelo apropriado. Operar vários modelos aumenta o custo, e as regras de seleção devem ser mantidas. A seleção do modelo, por si só, não garante que um texto editado preserve o contexto e a finalidade do artefato.

História de Usuário: SYS02-US03

"*Eu como usuário desejo salvar o documento completo*"

Critérios de Aceite:
[SYS02-US03-AC01] O sistema deverá compilar todas as partes do documento
[SYS02-US03-AC02] O sistema deverá corrigir eventuais erros gramaticais na versão salva
[SYS02-US03-AC03] O sistema deverá informar quais palavras foram corrigidas

Padrões e ADRs Aplicáveis:

Padrão [2]: Tubos e Filtros

Motivação: Um sistema precisa realizar uma variedade de tarefas de complexidade variável sobre os dados que processa.

Solução: Os dados são processados por meio de uma série de procedimentos, que têm entradas diferentes em vários estágios e produzem resultados incrementais à medida que ocorre a comparação entre as imagens de consulta e a imagem de entrada.

Consequências: O sistema se torna mais flexível; remover, modificar e adicionar filtros é mais fácil do que em um sistema monolítico

ADR: Processamento em estágios para salvar o documento completo Proposto

Contexto: Quando um usuário salva um documento, o sistema deve compilar todas as suas partes, corrigir quaisquer erros gramaticais na versão salva e informar ao usuário quais palavras foram corrigidas. Essas tarefas distintas devem produzir um documento salvo e um relatório de correções consistentes.

Decisão: Processaremos o documento por meio de estágios sequenciais que compilam suas partes, corrigem erros gramaticais enquanto registram quais palavras foram alteradas e, em seguida, salvam o documento corrigido e informam ao usuário as palavras corrigidas.

Consequências da Decisão: Separar as tarefas torna os estágios individuais mais fáceis de modificar do que uma única operação combinada. Os estágios devem permanecer em ordem, e as informações de correção devem ser transmitidas adiante para que as palavras informadas correspondam à versão salva.

Padrão [45]: Padrão Builder

Motivação: nan

Solução: Separar a construção de um objeto complexo de sua representação

Consequências: nan

ADR: Separar a montagem do documento do documento salvo

Proposto

Contexto: O usuário precisa salvar um documento de licitação completo, montado a partir de todas as suas partes. A versão salva deve ter os erros gramaticais corrigidos, e o usuário deve ser informado sobre quais palavras foram corrigidas. Esses requisitos exigem uma maneira de construir o documento completo, mantendo as correções relatadas consistentes com a versão salva.

Decisão: Separaremos a construção do documento completo de sua representação salva. O processo de construção compilará todas as partes do documento, produzirá a versão com a gramática corrigida para ser salva e identificará as palavras corrigidas para informar ao usuário.

Consequências da Decisão: O documento salvo conterá todas as partes compiladas e as correções gramaticais, com as palavras corrigidas disponíveis para serem informadas ao usuário. Separar a construção da representação salva introduz uma etapa de montagem e exige que as correções relatadas permaneçam consistentes com a versão que é salva.

Sistema: SYS03

Propósito: Processamento e validação de arquivos enviados para o sistema Comercial da Empresa por meio dos aplicativos integrados para solicitação de serviços

História de Usuário: SYS03-US01

"Como sistema IA, quero receber os arquivos do cliente, para que eu possa analisar os documentos enviados e verificar se a documentação pertence ao cliente solicitante."

Critérios de Aceite:
[SYS03-US01-AC01] O sistema IA deve receber todas as imagens enviadas pelo cliente.
[SYS03-US01-AC02] O sistema IA deve verificar se a foto do documento pertence ao cliente que enviou a foto selfie.

Padrões e ADRs Aplicáveis:

Padrão [1]: Encapsulando modelos de ML em salvaguardas baseadas em regras

Motivação: É impossível garantir a correção das previsões dos modelos de ML, portanto elas não devem ser usadas diretamente para funções relacionadas à segurança ou à proteção. Além disso, os modelos de ML podem ser instáveis e vulneráveis a ataques adversariais, ruído nos dados e deriva.

Solução: Introduza um mecanismo determinístico, baseado em regras, que decida o que fazer com os resultados das previsões, por exemplo, com base em verificações adicionais de qualidade.

Consequências: Risco reduzido de impactos negativos de previsões incorretas, mas uma arquitetura mais complexa.

ADR: Verificação de documento e selfie regida por regras

Proposto

Contexto: O sistema de IA deve receber todas as imagens enviadas pelo cliente e verificar se a foto do documento pertence ao cliente que enviou a selfie. Uma avaliação incorreta da IA poderia comprometer essa verificação de identidade. Como não é possível garantir a correção das previsões de IA, a arquitetura deve levar em conta essa incerteza ao determinar o resultado da verificação.

Decisão: Colocaremos salvaguardas determinísticas, baseadas em regras, entre a avaliação, pela IA, da foto do documento e da selfie e o resultado da verificação. As salvaguardas determinarão como lidar com o resultado da avaliação, de modo que a previsão da IA, por si só, não decidirá se a foto do documento pertence ao cliente.

Consequências da Decisão: As salvaguardas reduzirão o risco de que uma previsão incorreta da IA produza diretamente um resultado incorreto de verificação de identidade, mas não garantirão a correção. Elas também tornarão a arquitetura de verificação mais complexa.

História de Usuário: SYS03-US02

"*Como sistema IA, quero alterar o status do atendimento quando confirmar que o documento pertence ao cliente, para que o cliente possa receber a confirmação da validação e a Empresa seguir com o atendimento ao cliente.*"

Critérios de Aceite:
[SYS03-US02-AC01] O sistema IA deve se comunicar com o sistema Comercial da empresa e alterar o status do atendimento para PENDENTE DE EXECUÇÃO.
[SYS03-US02-AC02] Com a mudança de status, o sistema Comercial da empresa deve comunicar ao cliente.
[SYS03-US02-AC03] Com a mudança de status, o sistema Comercial da empresa deve seguir com a programação do atendimento.

Padrões e ADRs Aplicáveis:

Padrão [1]: Encapsulando modelos de ML em salvaguardas baseadas em regras

Motivação: É impossível garantir a correção das previsões dos modelos de ML, portanto elas não devem ser usadas diretamente para funções relacionadas à segurança ou à proteção. Além disso, os modelos de ML podem ser instáveis e vulneráveis a ataques adversariais, ruído nos dados e deriva.

Solução: Introduzir um mecanismo determinístico, baseado em regras, que decida o que fazer com os resultados das previsões, por exemplo, com base em verificações adicionais de qualidade.

Consequências: Risco reduzido de impactos negativos de previsões incorretas, mas uma arquitetura mais complexa.

ADR: Salvaguardas por regras para a alteração do status do atendimento Proposto

Contexto: A confirmação pela IA de que o documento pertence ao cliente desencadeia a alteração do status do atendimento para PENDENTE DE EXECUÇÃO. Essa alteração leva o sistema Comercial a comunicar o cliente e a seguir com a programação do atendimento. Como uma confirmação incorreta pode liberar essas etapas indevidamente, a arquitetura precisa controlar quando o resultado da IA pode produzir a mudança de status.

Decisão: Vamos submeter o resultado da confirmação de titularidade feita pela IA a uma decisão determinística baseada em regras antes de solicitar ao sistema Comercial a alteração do status para PENDENTE DE EXECUÇÃO. Somente um resultado autorizado por essas regras poderá desencadear a mudança de status; um resultado não autorizado não a desencadeará.

Consequências da Decisão: A decisão reduz o risco de que uma confirmação incorreta da IA provoque a comunicação ao cliente e a programação do atendimento. A comunicação e a programação permanecem vinculadas à mudança de status no sistema Comercial. Em contrapartida, a verificação por regras acrescenta complexidade à arquitetura e não elimina a possibilidade de erros na confirmação.

Padrão [3]: Cliente-Servidor

Motivação: Um número de usuários/aplicações requer dados de uma aplicação.

Solução: Um componente tem o papel de servidor e pelo menos um componente tem o papel de cliente, iniciando conexões para obter algum serviço.

Consequências: Os dados, assim como os periféricos de rede, são controlados centralmente. Uma desvantagem, porém, é que o servidor é caro para adquirir e gerenciar.

ADR: Sistema Comercial como Provedor de Serviço para Atualizações de Status Proposto

Contexto: Após confirmar que um documento pertence ao cliente, o sistema de IA deve alterar o status da solicitação de serviço para PENDENTE DE EXECUÇÃO. Essa alteração deve permitir que o sistema Comercial notifique o cliente e continue agendando o serviço. A arquitetura deve definir como o sistema de IA solicita a alteração enquanto o sistema Comercial mantém a solicitação de serviço.

Decisão: Faremos com que o sistema de IA atue como um cliente que solicita uma alteração de status ao sistema Comercial. Após confirmar que o documento pertence ao cliente, o sistema de IA solicitará que o sistema Comercial defina o status da solicitação de serviço como PENDENTE DE EXECUÇÃO. O sistema Comercial realizará a alteração, notificará o cliente e continuará agendando o serviço.

Consequências da Decisão: O status da solicitação de serviço e as ações que se seguem à sua alteração permanecem controlados centralmente pelo sistema Comercial. O sistema de IA depende desse sistema para concluir a alteração de status, e a manutenção desse serviço central acarreta um custo operacional.

Padrão [14]: Distinguir a lógica de negócios do modelo de ML

Motivação: Os sistemas de aprendizado de máquina (ML) são complexos porque seus componentes de ML precisam ser (re)treinados regularmente e têm um comportamento intrinsecamente não determinístico. Assim como em outros sistemas, os requisitos de negócios para esses sistemas, bem como os algoritmos de ML, mudam com o tempo.

Solução: Defina APIs claras entre componentes tradicionais e de ML. Coloque componentes de negócios e de ML com responsabilidades diferentes em três camadas. Divida os fluxos de dados em três.

Consequências: Desacoplar os componentes de negócios “tradicionais” dos componentes de ML permite que os componentes de ML sejam monitorados e ajustados para atender aos requisitos dos usuários e às entradas em mudança.

ADR: Separar a validação de documentos do fluxo de trabalho do Comercial Proposto

Contexto: Quando o sistema de AI confirma que um documento pertence ao cliente, ele deve alterar o status do serviço no sistema Comercial para PENDENTE DE EXECUÇÃO. Essa mudança de status permite que o sistema Comercial notifique o cliente e prossiga com o

agendamento. A arquitetura precisa distinguir a responsabilidade pela validação por AI dessas atividades de negócios subsequentes.

Decisão: Vamos separar a validação de documentos baseada em AI do fluxo de trabalho de negócios do sistema Comercial por meio de uma interface clara. Após confirmar que o documento pertence ao cliente, o sistema de AI se comunicará com o sistema Comercial para alterar o status do serviço para PENDENTE DE EXECUÇÃO. O sistema Comercial então cuidará da notificação ao cliente e do agendamento com base nessa mudança de status.

Consequências da Decisão: A separação permitirá que a validação por AI seja ajustada sem acoplá-la à notificação ao cliente ou ao agendamento. O sistema Comercial manterá a responsabilidade por essas atividades de negócios. O fluxo de trabalho dependerá de a validação confirmada ser comunicada e de a mudança de status ser bem-sucedida, portanto a interface entre os sistemas precisará ser mantida.

Padrão [44]: Padrão Estado

Motivação: nan

Solução: Permitir que um objeto altere seu comportamento quando seu estado interno mudar

Consequências: nan

ADR: Progressão da solicitação de serviço orientada por estado

Proposto

Contexto: Quando o sistema de IA confirmar que um documento pertence ao cliente, ele deve alterar o status da solicitação de serviço no sistema Comercial da empresa para PENDENTE DE EXECUÇÃO. Essa alteração deve levar o sistema Comercial a notificar o cliente e a prosseguir com o agendamento do serviço. A arquitetura precisa associar essas ações ao status da solicitação.

Decisão: Faremos com que o tratamento de uma solicitação de serviço pelo sistema Comercial dependa de seu estado. Após confirmar que o documento pertence ao cliente, o sistema de IA se comunicará com o sistema Comercial para alterar o status da solicitação para PENDENTE DE EXECUÇÃO. Nesse estado, o sistema Comercial notificará o cliente e prosseguirá com o agendamento do serviço.

Consequências da Decisão: A notificação do cliente e o agendamento seguirão a alteração confirmada de status, em vez de ocorrerem independentemente dela. Essas ações dependerão de o sistema de IA comunicar a alteração com sucesso e de o sistema Comercial manter o comportamento associado a PENDENTE DE EXECUÇÃO.

Padrão [50]: Padrão síncrono

Motivação: Quando, na sua lógica de negócios, a inferência do modelo é um bloqueio para prosseguir para a próxima etapa

Solução: bloqueia o fluxo de trabalho do sistema até que a predição termine

Consequências: Fácil de gerenciar devido à sua simplicidade. Todos os aspectos operacionais, como rastreamento de transações, monitoramento etc., também se tornam fáceis. O fluxo de trabalho do serviço se torna simples, pois o processo não prosseguirá até que a predição seja concluída. Mas: (1) A latência da predição pode se tornar um gargalo de desempenho. (2) Talvez você tenha que considerar uma solução alternativa para não degradar a experiência do usuário por causa da latência da predição. (3) Se o cliente do seu serviço for outro serviço, então este padrão leva a threads bloqueadas no lado do cliente.

ADR: Validação Síncrona de Documentos Antes da Mudança de Status Proposto

Contexto: O sistema de IA deve confirmar que um documento pertence ao cliente antes de alterar o status da solicitação de serviço. A notificação ao cliente e o agendamento do serviço pelo sistema Comercial dependem dessa mudança de status, portanto, o fluxo de trabalho não deve avançar antes que a validação esteja concluída.

Decisão: Manteremos o fluxo de trabalho da solicitação de serviço em espera até que o sistema de IA conclua a validação do documento. Quando a validação confirmar que o documento pertence ao cliente, o sistema de IA se comunicará com o sistema Comercial para mudar o status para PENDENTE DE EXECUÇÃO, permitindo que o sistema Comercial notifique o cliente e prossiga com o agendamento.

Consequências da Decisão: O fluxo de trabalho preservará a ordem exigida entre validação, mudança de status, notificação e agendamento.

Essa sequência é simples de acompanhar, mas a latência da validação pode atrasar a mudança de status e todas as etapas subsequentes.

História de Usuário: SYS03-US03

"Como sistema IA, quero baixar o serviço como improcedente quando confirmar que o documento não pertence ao cliente, para que o cliente possa receber uma notificação da Empresa informando que não foi possível confirmar o cliente."

Critérios de Aceite:
[SYS03-US03-AC01] O sistema IA deve baixar o serviço no sistema Comercial da empresa como improcedente e preencher o motivo da improcedência para ser informado ao cliente.
[SYS03-US03-AC02] Com a baixa do serviço, o sistema Comercial da empresa deve notificar o cliente.

Padrões e ADRs Aplicáveis:

Padrão [3]: Cliente-Servidor

Motivação: Um número de usuários/aplicações requer dados de uma aplicação.

Solução: Um componente tem um papel de servidor e pelo menos um componente tem o papel de cliente, iniciando conexões a fim de obter algum serviço.

Consequências: Os dados, assim como os periféricos de rede, são controlados centralmente. Uma desvantagem, no entanto, é que o servidor é caro para adquirir e gerenciar.

ADR: Sistema Comercial como Servidor para Encerramento de Serviço Proposto

Contexto: Quando o sistema de IA confirma que um documento enviado não pertence ao cliente, ele deve marcar o serviço como improcedente e registrar o motivo no sistema Comercial da empresa. O sistema Comercial deve então notificar o cliente. A arquitetura precisa definir como o sistema de IA solicita essa atualização, deixando a notificação a cargo do sistema Comercial.

Decisão: Tornaremos o sistema de IA um cliente do sistema Comercial, que atenderá às solicitações de atualização de serviços. Quando o sistema de IA confirmar que um documento não pertence ao cliente, ele solicitará

que o sistema Comercial marque o serviço como improcedente e registre o motivo. O sistema Comercial notificará o cliente quando o serviço for encerrado.

Consequências da Decisão: O encerramento do serviço e seu motivo serão registrados no sistema Comercial, que permanece responsável por notificar o cliente. Isso centraliza o controle do registro do serviço, mas faz com que a conclusão da atualização dependa de o sistema Comercial estar disponível e ser mantido.

Padrão [50]: Padrão síncrono

Motivação: Quando, na sua lógica de negócios, a inferência do modelo é um impedimento para prosseguir para a próxima etapa

Solução: bloqueia o fluxo de trabalho do sistema até que a predição termine

Consequências: Fácil de gerenciar devido à sua simplicidade. Todos os aspectos operacionais, como rastreamento de transações, monitoramento etc., também se tornam fáceis. O fluxo de trabalho do serviço se torna simples, pois o processo não prosseguirá até que a predição seja concluída. Mas: (1) A latência da predição pode se tornar um gargalo de desempenho. (2) Talvez você tenha que considerar uma solução alternativa para não degradar a experiência do usuário por causa da latência da predição. (3) Se o cliente do seu serviço for outro serviço, então esse padrão leva a threads bloqueadas no lado do cliente.

ADR: Verificação síncrona de documentos antes do encerramento do serviço Proposto

Contexto: O sistema de IA processa documentos enviados por meio de aplicativos integrados de solicitação de serviço. Ele deve marcar um serviço como improcedente no sistema Comercial da empresa, com um motivo, quando confirma que o documento não pertence ao cliente. O sistema Comercial então notifica o cliente. Portanto, a atualização do serviço depende do resultado da verificação do documento.

Decisão: Aguardaremos a conclusão da verificação do documento pela IA antes de prosseguir com a atualização do serviço. Se a verificação confirmar que o documento não pertence ao cliente, marcaremos o serviço como improcedente no sistema Comercial e registraremos o motivo, permitindo que o sistema Comercial notifique o cliente após o encerramento do serviço.

Consequências da Decisão: O fluxo de trabalho do serviço terá uma sequência clara: verificação do documento, encerramento condicional do serviço e, em seguida, notificação. Isso torna o fluxo de trabalho mais simples de acompanhar e rastrear. A espera pelo resultado da IA pode atrasar o encerramento do serviço e a notificação subsequente se a verificação demorar muito.

Sistema: SYS04

Propósito: Reproduzir digitalmente a estrutura física, os sensores, os equipamentos e o ambiente de automação da ETA, permitindo executar testes, simular condições operacionais e avaliar estratégias de controle sem interferir na operação real.

História de Usuário: SYS04-US02

"Como desenvolvedor de automação, quero que o simulador disponibilize variáveis e receba comandos por meio dos protocolos definidos no projeto, para testar a integração com CLP, supervisório e aplicações externas."

Critérios de Aceite:
[SYS04-US02-AC01] O sistema deve disponibilizar as variáveis simuladas por OPC UA.
[SYS04-US02-AC02] As tags devem seguir a nomenclatura e os tipos de dados definidos para a ETA.
[SYS04-US02-AC03] O sistema deve receber comandos permitidos e refletir seus efeitos na simulação.
[SYS04-US02-AC04] A perda e o restabelecimento da comunicação devem poder ser testados.

Padrões e ADRs Aplicáveis:

Padrão [3]: Cliente-Servidor

Motivação: Um número de usuários/aplicações requer dados de uma aplicação.

Solução: Um componente tem um papel de servidor e pelo menos um componente tem o papel de cliente, iniciando conexões para obter algum serviço.

Consequências: Os dados, bem como os periféricos de rede, são controlados centralmente. Uma desvantagem, porém, é que o servidor é caro para adquirir

e gerenciar.

> **ADR: Interface Cliente-Servidor para Integração do Simulador**
>
> Proposto
>
> **Contexto:** O desenvolvedor de automação precisa testar a integração do simulador com CLP, supervisório e aplicações externas. Para isso, essas aplicações precisam acessar variáveis simuladas por OPC UA, com a nomenclatura e os tipos de dados definidos para a ETA, e enviar comandos permitidos cujos efeitos apareçam na simulação. Também deve ser possível testar a perda e o restabelecimento da comunicação.
>
> **Decisão:** Estabeleceremos o simulador como o servidor para esses serviços de integração, com o CLP, o sistema supervisório e as aplicações externas iniciando conexões como clientes. O simulador disponibilizará as variáveis simuladas por OPC UA conforme a nomenclatura e os tipos de dados definidos para a ETA, receberá comandos permitidos e refletirá seus efeitos na simulação. As conexões com os clientes permitirão testar a perda e o restabelecimento da comunicação.
>
> **Consequências da Decisão:** A disponibilização das variáveis e o recebimento de comandos ficarão concentrados no simulador, oferecendo aos clientes uma interface comum para os testes de integração. Essa concentração também torna a comunicação dos clientes dependente da disponibilidade do servidor e exige esforço para mantê-lo e gerenciá-lo. A perda dessa comunicação interromperá as trocas com os clientes até seu restabelecimento, comportamento que poderá ser testado.

Padrão [28]: Testes de ponta a ponta

Motivação: Não se deve permitir que um modelo ou pipeline de ML com falhas seja lançado e produza resultados ruins no aplicativo.

Solução: Testes de fumaça manuais são sempre úteis, e manter os testes atualizados com novos recursos, casos de uso e dados é uma tarefa contínua. As previsões de modelos de ML não são diferentes. Se uma parte do aplicativo é atendida por uma recomendação de um modelo de ML, identifique as asserções que podem ser feitas

Consequências: nan

ADR: Testes de integração de ponta a ponta do simulador

Proposto

Contexto: Os desenvolvedores de automação precisam testar a integração do simulador com o PLC, o sistema supervisório e as aplicações externas. Isso exige verificar se as variáveis simuladas estão disponíveis por meio de OPC UA com os nomes de tags e tipos de dados definidos pela ETA, se os comandos permitidos afetam a simulação e se a perda e o restabelecimento da comunicação podem ser testados.

Decisão: Estabeleceremos testes de ponta a ponta das interfaces de integração do simulador. Esses testes verificarão as variáveis OPC UA e seus nomes de tags e tipos de dados, executarão comandos permitidos e observarão seus efeitos na simulação, e abrangerão a perda e o restabelecimento da comunicação.

Consequências da Decisão: Os testes fornecerão uma maneira de detectar discrepâncias de integração nos comportamentos que os desenvolvedores precisam avaliar. Eles também exigirão manutenção contínua à medida que as tags, os comandos ou o comportamento de integração definidos mudarem.

História de Usuário: SYS04-US03

"Como cientista de dados ou engenheiro de controle, quero conectar modelos de controle e inteligência artificial ao gêmeo digital, para avaliar suas recomendações antes da aplicação na ETA real."

Critérios de Aceite:
[SYS04-US03-AC01] O sistema deve fornecer aos modelos as mesmas variáveis previstas para o ambiente real.
[SYS04-US03-AC02] Os comandos ou recomendações dos modelos devem ser aplicados somente ao ambiente simulado.
[SYS04-US03-AC03] O sistema deve registrar entradas, saídas e resultados de cada execução.
[SYS04-US03-AC04] Deve ser possível comparar diferentes modelos utilizando o mesmo cenário.

Padrões e ADRs Aplicáveis:

Padrão [1]: Encapsulando Modelos de ML em Salvaguardas Baseadas em Regras

Motivação: É impossível garantir a correção das previsões de modelos de ML, portanto elas não devem ser usadas diretamente para funções relacionadas à segurança ou à proteção. Além disso, os modelos de ML podem ser instáveis e vulneráveis a ataques adversariais, ruído nos dados e deriva.

Solução: Introduzir um mecanismo determinístico, baseado em regras, que decida o que fazer com os resultados das previsões, por exemplo, com base em verificações adicionais de qualidade.

Consequências: Risco reduzido de impactos negativos de previsões incorretas, mas uma arquitetura mais complexa.

> **ADR: Salvaguardas Baseadas em Regras para Avaliação de Modelos** Proposto
>
> **Contexto:** Cientistas de dados e engenheiros de controle precisam avaliar as recomendações dos modelos de controle e de IA no gêmeo digital antes de aplicá-las à ETA real. Os modelos devem receber as variáveis esperadas no ambiente real, mas seus comandos e recomendações podem ser aplicados apenas em simulação. Cada execução deve ser registrada, e diferentes modelos devem poder ser comparados sob o mesmo cenário. Como não se pode presumir que as saídas dos modelos estejam corretas, a arquitetura precisa preservar o limite entre a avaliação e a operação real.
>
> **Decisão:** Colocaremos uma salvaguarda determinística, baseada em regras, entre os modelos conectados e a aplicação de seus comandos ou recomendações. A salvaguarda permitirá que essas saídas afetem apenas o ambiente simulado e impedirá sua aplicação à ETA real. As avaliações com salvaguarda fornecerão aos modelos as variáveis esperadas no ambiente real e registrarão as entradas, saídas e resultados de cada execução para que diferentes modelos possam ser comparados sob o mesmo cenário.
>
> **Consequências da Decisão:** As saídas dos modelos podem ser avaliadas sem afetar diretamente a ETA real, reduzindo o risco decorrente de recomendações incorretas. As execuções registradas e os cenários compartilhados permitem a comparação de modelos. A salvaguarda acrescenta complexidade arquitetural, e suas regras devem permanecer coerentes com o limite de aplicação apenas em simulação.

Padrão [3]: Cliente-Servidor

Motivação: Um certo número de usuários/aplicações requer dados de uma aplicação.

Solução: Um componente tem o papel de servidor e pelo menos um componente tem o papel de cliente, iniciando conexões para obter algum serviço.

Consequências: Os dados, assim como os periféricos de rede, são controlados centralmente. Uma desvantagem, no entanto, é que o servidor é caro para adquirir e gerenciar.

> **ADR: Gêmeo Digital como Servidor de Simulação** Proposto
>
> **Contexto:** Modelos de controle e de AI precisam se conectar ao gêmeo digital para avaliar suas recomendações antes da aplicação na estação real de tratamento de água. Cada modelo precisa das variáveis esperadas no ambiente real, enquanto seus comandos ou recomendações devem afetar apenas a simulação. As avaliações devem registrar entradas, saídas e resultados e permitir que diferentes modelos sejam comparados sob o mesmo cenário.
>
> **Decisão:** Faremos do gêmeo digital o servidor para as avaliações de modelos, com os modelos conectados atuando como clientes que solicitam as variáveis esperadas e enviam comandos ou recomendações para simulação. O servidor aplicará esses comandos ou recomendações apenas no ambiente simulado, registrará as entradas, saídas e resultados de cada execução e fornecerá o mesmo cenário para comparações entre modelos.
>
> **Consequências da Decisão:** A centralização dos dados e da execução da simulação dá aos modelos conectados uma base consistente para comparação e mantém suas avaliações separadas da estação real. Isso também coloca no servidor do gêmeo digital a responsabilidade de atender aos modelos, executar avaliações e manter registros de execução, o que exigirá recursos de gestão.

Padrão [14]: Distinguir a lógica de negócios do modelo de ML

Motivação: Os sistemas de aprendizado de máquina (ML) são complexos porque seus componentes de ML devem ser (re)treinados regularmente e têm um comportamento intrinsecamente não determinístico. Assim como em outros sistemas, os requisitos de negócio para esses sistemas, bem como os algoritmos de ML, mudam com o tempo.

Solução: Defina APIs claras entre os componentes tradicionais e os componentes de ML. Coloque os componentes de negócio e de ML com responsabilidades diferentes em três camadas. Divida os fluxos de dados em três.

Consequências: A dissociação dos componentes de negócio “tradicionais” e dos componentes de ML permite que os componentes de ML sejam monitorados e ajustados para atender aos requisitos dos usuários e às mudanças nas entradas.

ADR: Separar os modelos da simulação e avaliação do gêmeo digital Proposto

Contexto: Cientistas de dados e engenheiros de controle precisam conectar modelos de controle e de IA ao gêmeo digital para avaliar suas recomendações antes de aplicá-las na ETA real. Os modelos devem receber as variáveis planejadas para o ambiente real, enquanto seus comandos ou recomendações devem afetar apenas a simulação. Cada execução deve ser registrada, e diferentes modelos devem poder ser comparados sob o mesmo cenário. Essas necessidades exigem uma fronteira clara entre o comportamento dos modelos e as funções de simulação e avaliação do gêmeo.

Decisão: Separaremos a simulação, os modelos conectados e a avaliação das execuções em três camadas, com interfaces explícitas entre os modelos e as outras funções. A camada de simulação fornecerá as variáveis necessárias e aplicará os comandos ou recomendações dos modelos apenas no ambiente simulado. A camada de avaliação registrará as entradas, saídas e resultados de cada execução e permitirá a comparação de diferentes modelos usando o mesmo cenário. Os dados fluirão separadamente da simulação para os modelos como entradas, dos modelos para a simulação como comandos ou recomendações, e das execuções para a avaliação como resultados registrados.

Consequências da Decisão: Os modelos podem ser alterados e comparados sem combinar seu comportamento com as responsabilidades de simulação e avaliação do gêmeo. As execuções registradas disponibilizarão suas recomendações e seus resultados para avaliação antes do uso na ETA real. A separação também exige a manutenção das interfaces e a coordenação dos três fluxos de dados para que os modelos recebam variáveis consistentes e as comparações usem o mesmo cenário.

Padrão [15]: Delegação da Responsabilidade pela Segurança

Motivação: Desenvolvimentos recentes em ML e DL permitiram que sistemas probabilísticos com grandes espaços de entrada e saída fossem explorados em sistemas críticos para a segurança. Isso introduz desafios relacionados à segurança, como: (1) Complexidade e opacidade do modelo, (2) Lidar com saídas probabilísticas, (3) Sensibilidade a mudanças na distribuição (4) A verificação formal é impossível ou não escalável e mais. Essa responsabilidade pela segurança precisa ser tratada.

Solução: Delegar toda a responsabilidade pela segurança a outros componentes ou envolver algoritmos de DL em limites de segurança.

Consequências: Maior segurança, ao mesmo tempo que se impõem restrições limitadas ao componente de DL. Mas maior complexidade.

ADR: Fronteira Restrita à Simulação para Saídas de Modelos

Proposto

Contexto: Cientistas de dados e engenheiros de controle precisam avaliar recomendações de modelos de controle e de IA no gêmeo digital antes de aplicá-las à ETA real. Os modelos devem receber as variáveis esperadas no ambiente real, mas seus comandos ou recomendações devem afetar apenas a simulação. Cada execução também deve ser registrada para que diferentes modelos possam ser comparados usando o mesmo cenário.

Decisão: Estabeleceremos uma fronteira em torno dos modelos conectados que torna o gêmeo digital, e não os modelos, responsável por limitar seus efeitos ao ambiente simulado. A fronteira fornecerá as variáveis esperadas aos modelos, aplicará seus comandos ou recomendações apenas na simulação e registrará as entradas, saídas e resultados de cada execução para comparação sob o mesmo cenário.

Consequências da Decisão: Os modelos podem ser avaliados sem que suas saídas afetem a ETA real, e as execuções registradas permitirão comparações sob um cenário comum. A fronteira acrescenta complexidade arquitetural e deve ser mantida à medida que os modelos são conectados. Os resultados obtidos na simulação não estabelecem, por si só, que uma recomendação seja segura para aplicação à ETA real.

Padrão [28]: Testes de ponta a ponta

Motivação: Não se deve permitir que um modelo ou pipeline de ML defeituoso seja lançado e produza resultados ruins no aplicativo.

Solução: Os testes de fumaça manuais são sempre úteis, e manter os testes renovados e atualizados com novos recursos, casos de uso e dados é uma tarefa contínua. As previsões de modelos de ML não são diferentes. Se uma parte do aplicativo é atendida por uma recomendação de um modelo de ML, identifique as asserções que podem ser feitas

Consequências: nan

ADR: Avaliação de modelos de ponta a ponta no gêmeo digital

Proposto

Contexto: Cientistas de dados e engenheiros de controle precisam avaliar as recomendações de modelos de controle e de IA no gêmeo digital antes de aplicá-las à ETA real. Os modelos devem receber as variáveis esperadas no ambiente real, enquanto seus comandos ou recomendações afetam apenas a simulação. Cada execução deve ser registrada, e diferentes modelos devem ser comparáveis sob o mesmo cenário.

Decisão: Avaliaremos os modelos conectados por meio de execuções de testes de ponta a ponta no gêmeo digital. Cada execução fornecerá as variáveis esperadas, aplicará comandos ou recomendações dos modelos apenas no ambiente simulado e registrará entradas, saídas e resultados. Usaremos asserções sobre o comportamento observado e executaremos diferentes modelos no mesmo cenário para comparação.

Consequências da Decisão: As execuções dos testes tornarão o comportamento dos modelos observável e comparável antes que as recomendações sejam consideradas para a ETA real. Manter as asserções e os cenários de teste relevantes à medida que os modelos e os casos de uso mudam exigirá esforço contínuo. Passar em um teste simulado mostrará como um modelo se comportou naquele cenário, não garantirá seu comportamento na ETA real.

Padrão [36]: Reprodução

Motivação: Necessidade de uma forma de testar modelos e dados operacionais.

Solução: Investir em um framework de testes em lote.

Consequências: nan

ADR: Testes de Cenários Repetíveis para Modelos de Controle e AI

Proposto

Contexto: Cientistas de dados e engenheiros de controle precisam avaliar as recomendações dos modelos no gêmeo digital antes de aplicá-las à estação real de tratamento de água. Os modelos devem receber as variáveis esperadas no ambiente real, enquanto seus comandos e recomendações afetam apenas a simulação. Cada execução deve ser registrada, e diferentes modelos devem ser comparáveis sob o mesmo cenário.

Decisão: Estabeleceremos testes em lote para o gêmeo digital que reproduzam o mesmo cenário simulado para diferentes modelos de controle e AI. Cada execução fornecerá ao modelo as variáveis esperadas no ambiente real, aplicará seus comandos ou recomendações apenas à simulação e registrará suas entradas, saídas e resultados.

Consequências da Decisão: Cenários repetidos permitirão comparações entre modelos antes que suas recomendações sejam aplicadas à planta real. Os comandos e as recomendações dos modelos feitos durante os testes permanecerão restritos à simulação. A capacidade de testes exigirá a manutenção de cenários e registros comparáveis para cada execução; seus resultados descreverão o desempenho na simulação, não estabelecerão o desempenho na planta real.

Padrão [43]: Padrão Estratégia

Motivação: Como um modelo de ML que executa uma tarefa em um determinado contexto pode ser alterado de maneira flexível?

Solução: Defina uma interface (estratégia) que diferentes modelos implementem. O contexto chamará os métodos expostos pela interface, e os modelos implementados se comportarão de maneira diferente com base nos dados contextuais.

Consequências: Trocar modelos ou alcançar flexibilidade no comportamento do modelo é mais fácil, mas a complexidade do código aumenta.

ADR: Modelos intercambiáveis para avaliação do gêmeo digital

Proposto

Contexto: Cientistas de dados e engenheiros de controle precisam conectar diferentes modelos de controle e de AI ao gêmeo digital e comparar suas recomendações antes de aplicá-las à ETA real. Cada modelo deve receber as variáveis esperadas no ambiente real, enquanto seus comandos ou recomendações devem afetar apenas a simulação. Cada execução também deve ser registrada, e os modelos devem ser comparáveis usando o mesmo cenário.

Decisão: Definiremos uma interface comum para modelos de controle e de AI que o gêmeo digital usará para invocar modelos intercambiáveis. O gêmeo fornecerá a cada modelo as variáveis necessárias para um cenário, aplicará seus comandos ou recomendações apenas ao ambiente simulado e registrará as entradas, saídas e resultados de cada execução. Essa estrutura permitirá que diferentes modelos sejam avaliados em relação ao mesmo cenário.

Consequências da Decisão: Os modelos podem ser trocados e comparados sem alterar a forma como o gêmeo digital os invoca. Suas recomendações podem ser avaliadas sem afetar a ETA real, e as execuções registradas apoiarão a comparação. Cada modelo deve estar em conformidade com a interface comum, o que aumenta a complexidade do código.